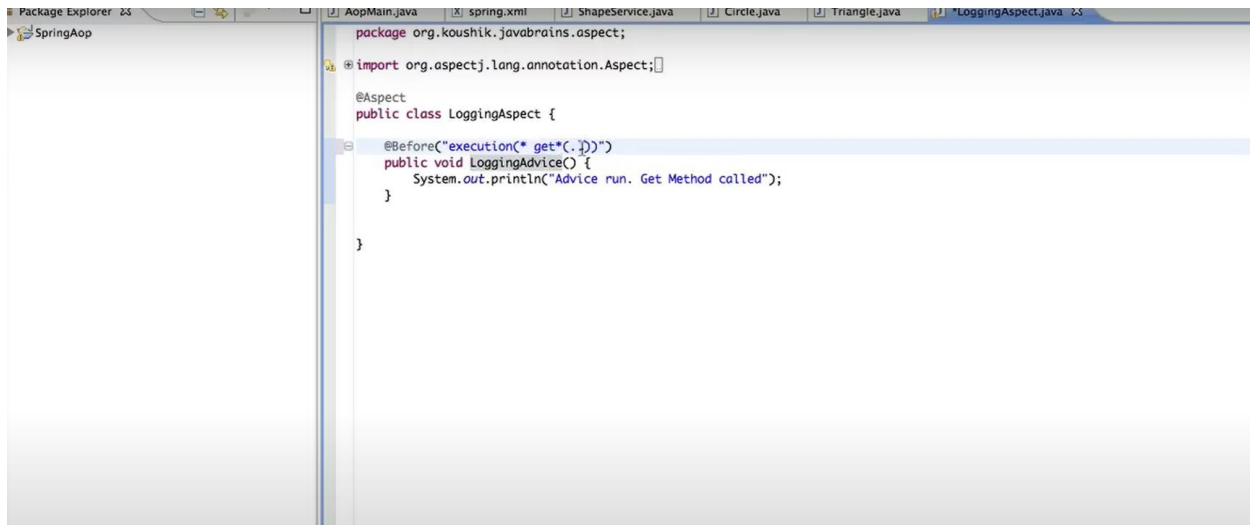


@Aspect :



```
package org.koushik.javabrainz.aspect;

import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {

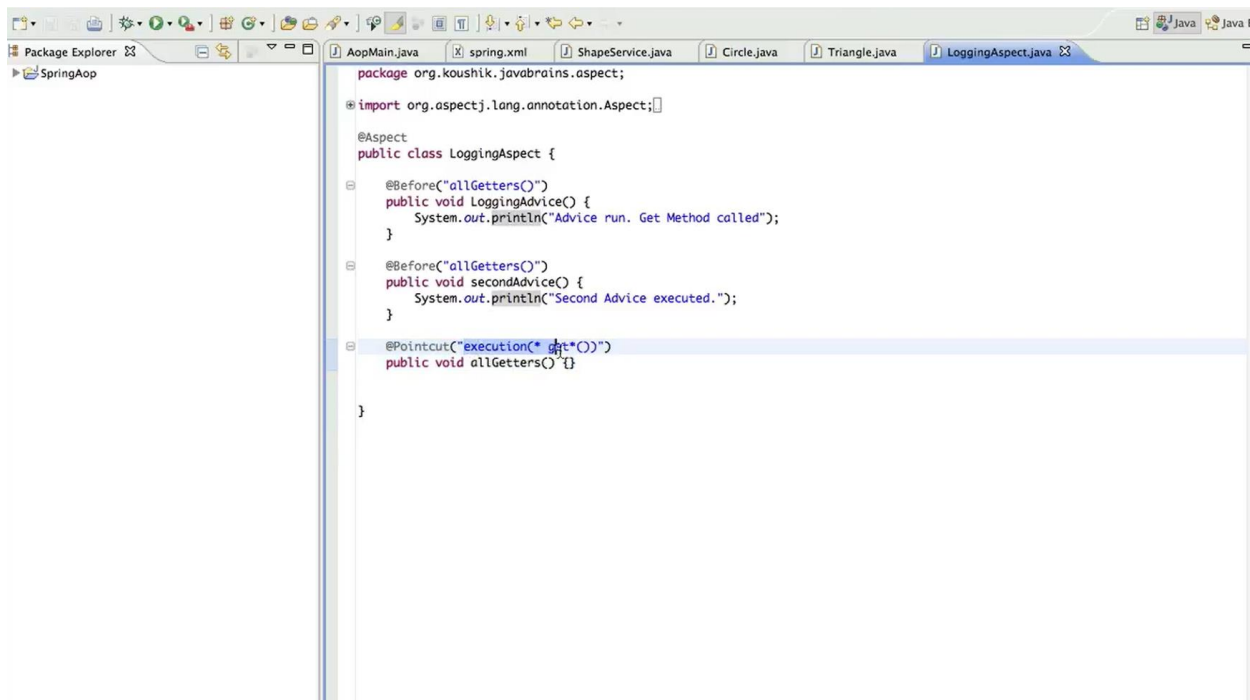
    @Before("execution(* get*(..))")
    public void LoggingAdvice() {
        System.out.println("Advice run. Get Method called");
    }

}
```

If we want to apply all zero arguments and any arguments methods – use inside method (..)

*-is a wild card entry , it will replace with any name can come.

@PointCut:



```
package org.koushik.javabrainz.aspect;

import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {

    @Before("allGetters()")
    public void LoggingAdvice() {
        System.out.println("Advice run. Get Method called");
    }

    @Before("allGetters()")
    public void secondAdvice() {
        System.out.println("Second Advice executed.");
    }

    @Pointcut("execution(* get*())")
    public void allGetters() {}

}
```

@Pointcut will help us to not to repeat similar type of execution method name sin many places

How @Before(execution(* * * * *)) works :

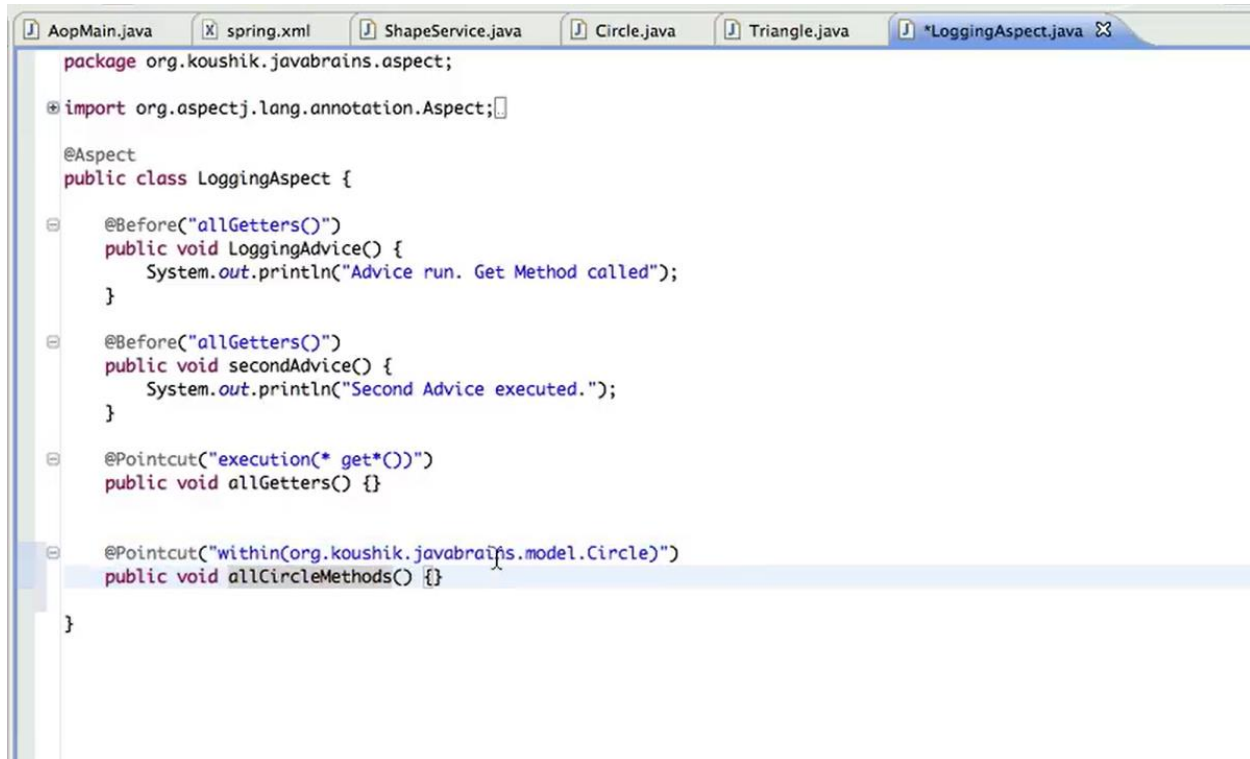
Public String getName(int id); -----→ we must provide these details for execution method - * * * (..)

Point cut Expressions:

1. execution

@Before(execution(public * org.javabrainz.harikrishna.Circle.*get(..))

2. within – with specific class or package if you want to run the advice for all the methods before executing.



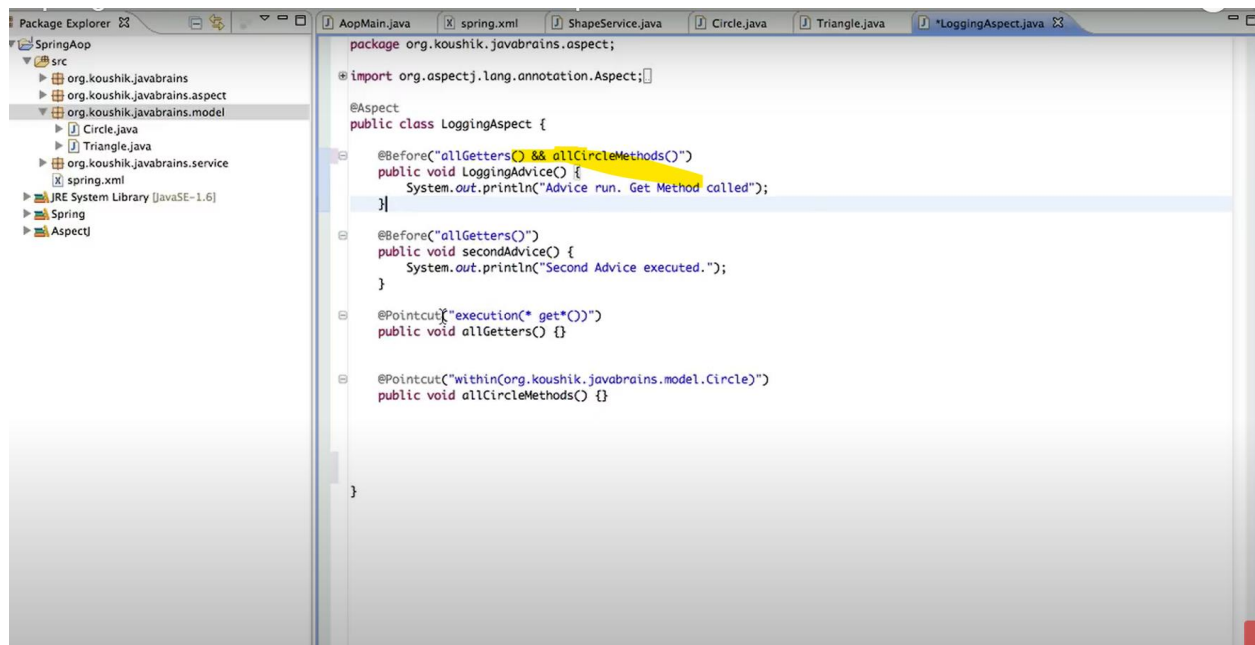
Here also we can apply wild card entries *, within this package and classes all methods also

And if you want to apply same package and inside sub packages also, then we have to give two (..) end of the wilds card

3. args – spring will look at all the methods which matches given arguments, then it will execute for all those

@Pointcut(args())

Combine the Pointcut Expressions:



```
import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {

    @Before("allGetters()")
    public void LoggingAdvice() {
        System.out.println("Advice run. Get Method called");
    }

    @Before("allGetters()")
    public void secondAdvice() {
        System.out.println("Second Advice executed.");
    }

    @Pointcut("execution(* get*())")
    public void allGetters() {}

    @Pointcut("within(org.koushik.javabrains.model.*)")
    public void allCircleMethods() {}

}
```

JoinPoint – This annotation will be used to capture/pass the input of the method to the advised method inside Aspect class

```
package org.koushik.javabrainz.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.koushik.javabrainz.model.Circle;

@Aspect
public class LoggingAspect {

    @Before("allCircleMethods()")
    public void LoggingAdvice(JoinPoint joinPoint) {

    }

    @Before("args(name)")
    public void stringArgumentMethods(String name) {
        System.out.println("A method that takes String arguments has been called. The value is " + name);
    }

    @Pointcut("execution(* get*())")
    public void allGetters() {}

    @Pointcut("within(org.koushik.javabrainz.model.Circle)")
    public void allCircleMethods() {}

}
```

```
package org.koushik.javabrainz.aspect;

import org.aspectj.lang.JoinPoint;

@Aspect
public class LoggingAspect {

    @Before("allCircleMethods()")
    public void LoggingAdvice(JoinPoint joinPoint) {

    }

    @AfterReturning("args(name)")
    public void stringArgumentMethods(String name) {
        System.out.println("A method that takes String arguments has been called. The value is " + name);
    }

    @AfterThrowing("args(name)")
    public void exceptionAdvice(String name) {
        System.out.println("An exception has been thrown");
    }

    @Pointcut("execution(* get*())")
    public void allGetters() {}

    @Pointcut("within(org.koushik.javabrainz.model.Circle)")
    public void allCircleMethods() {}

}
```

